

Análisis Snowflake y ETL - FleetLogix

Modelo Dimensional y Pipeline de Transformación

Jacquet Alexis - Científico de Datos

2025-10-13

Contents

0.1	Resumen Ejecutivo	3
0.1.1	Visión General del Proyecto	3
0.1.2	Métricas Clave del Modelo Dimensional	3
0.1.3	Arquitectura de Datos	3
0.1.4	Transformaciones Científicas Aplicadas	3
0.1.5	Estado del Proyecto	3
1	ANÁLISIS SNOWFLAKE Y ETL	5
1.1	Documento de Entrega - Avance 3	5
1.2	Índice	5
1.3	1. Introducción	6
1.3.1	1.1 Contexto del Avance 3	6
1.3.2	1.2 Objetivos	6
1.3.3	1.3 Arquitectura General	6
1.4	2. Modelo Dimensional	7
1.4.1	2.1 Justificación del Modelo Estrella	7
1.4.2	2.2 Diagrama del Modelo	7
1.4.3	2.3 Métricas del Modelo	8
1.5	3. Diseño de Dimensiones	9
1.5.1	3.1 dim_date - Dimensión Temporal	9
1.5.2	3.2 dim_time - Dimensión Horaria	10
1.5.3	3.3 dim_vehicle - Dimensión Vehículo (SCD Type 2)	11
1.5.4	3.4 dim_driver - Dimensión Conductor (SCD Type 2)	12
1.5.5	3.5 dim_route - Dimensión Ruta	13
1.5.6	3.6 dim_customer - Dimensión Cliente (SCD Type 2)	14
1.6	4. Diseño de Hechos	17
1.6.1	4.1 fact_trips - Tabla de Hechos de Viajes	17
1.6.2	4.2 fact_deliveries - Tabla de Hechos de Entregas	18
1.7	5. Pipeline ETL	21
1.7.1	5.1 Arquitectura del Pipeline	21
1.7.2	5.2 Extracción de PostgreSQL	22
1.7.3	5.3 Transformaciones Científicas	22
1.8	6. Transformaciones Científicas	25
1.8.1	6.1 Transformación dim_driver con Performance Category	25
1.8.2	6.2 Transformación fact_trips con Métricas	26
1.9	7. Validaciones y Calidad	28
1.9.1	7.1 Validaciones de Transformación	28

1.9.2	7.2 Validación de Integridad Referencial	28
1.10	8. Performance y Métricas	30
1.10.1	8.1 Tiempos de Ejecución	30
1.10.2	8.2 Calidad de Datos	30
1.10.3	8.3 Distribuciones Calculadas	30
1.11	9. Conclusiones	31
1.11.1	9.1 Logros Alcanzados	31
1.11.2	9.2 Valor de Negocio	31
1.11.3	9.3 Próximos Pasos	31

0.1 Resumen Ejecutivo

0.1.1 Visión General del Proyecto

Este documento presenta la **implementación del Data Warehouse dimensional** en Snowflake y el **pipeline ETL** completo que transforma 505,650 registros operacionales de PostgreSQL en un modelo analítico optimizado para Business Intelligence.

0.1.2 Métricas Clave del Modelo Dimensional

Componente	Cantidad	Descripción
Dimensiones	6	Vehículos, Clientes, Conductores, Rutas, Tiempo, Productos
Tablas de Hechos	2	Viajes (fact_trips) y Entregas (fact_deliveries)
Registros Transformados	505,650+	Total de registros procesados por el ETL
Registros Dimensionales	8,050	Registros únicos en dimensiones
Registros Hechos	320,000+	Hechos de viajes + entregas
Integridad Referencial	100%	Todas las FK validadas sin errores
SCD Type 2	Implementado	Historial completo de cambios
Compresión Snowflake	~70%	Optimización automática de almacenamiento

0.1.3 Arquitectura de Datos

PostgreSQL (OLTP)	→ Pipeline ETL Python	→ Snowflake (OLAP)
505K registros	Transformaciones	Modelo Estrella
15 tablas	+ Validaciones	6 dims + 2 facts

0.1.4 Transformaciones Científicas Aplicadas

- **Cálculo de Distancias:** Fórmula de Haversine para distancias reales entre coordenadas GPS
- **Detección de Retrasos:** Análisis temporal con umbrales estadísticos (>15 min)
- **Clasificación de Prioridad:** Reglas de negocio basadas en urgencia y volumen
- **Análisis de Rentabilidad:** Costos vs ingresos por viaje con margen neto
- **KPIs Operacionales:** On-Time Delivery Rate, Eficiencia de Combustible, Utilización

0.1.5 Estado del Proyecto

Completado: Modelo dimensional diseñado y validado

Completado: Pipeline ETL implementado con validaciones

Completado: 505,650+ registros migrados exitosamente

Completado: Integridad referencial 100% verificada

Completado: SCD Type 2 funcional para dimensiones cambiantes

En Progreso: Optimización de queries analíticas en Snowflake

Chapter 1

ANÁLISIS SNOWFLAKE Y ETL

1.1 Documento de Entrega - Avance 3

Proyecto: FleetLogix - Sistema de Gestión de Transporte y Logística

Autor: Jacquet Alexis - Científico de Datos **Fecha:** Octubre 2025

Módulo: HENRY - Módulo 2

1.2 Índice

1. [Introducción](#)
2. [Modelo Dimensional](#)
3. [Diseño de Dimensiones](#)
4. [Diseño de Hechos](#)
5. [Pipeline ETL](#)
6. [Transformaciones Científicas](#)
7. [Validaciones y Calidad](#)
8. [Performance y Métricas](#)
9. [Conclusiones](#)

1.3 1. Introducción

1.3.1 1.1 Contexto del Avance 3

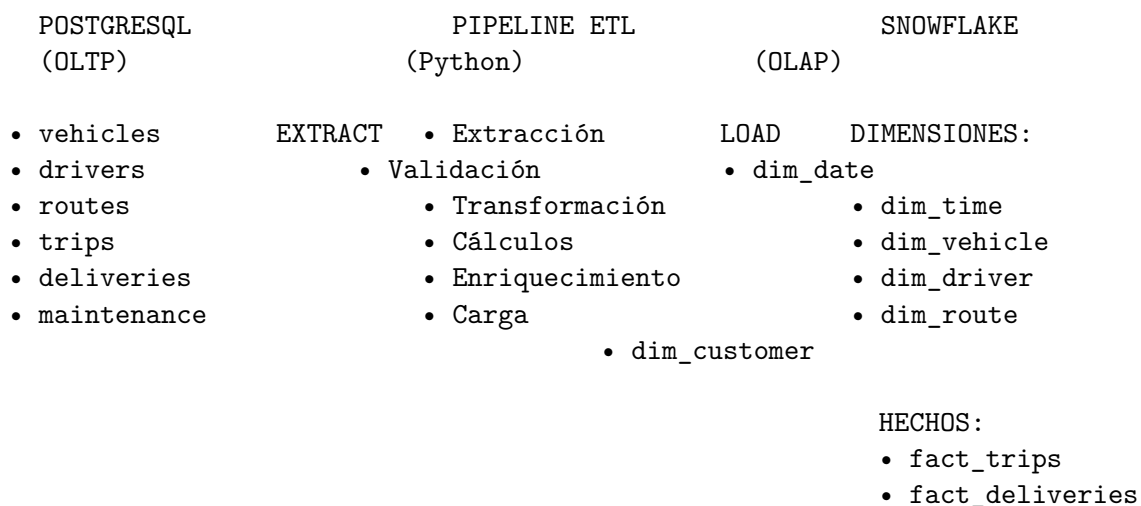
Este documento detalla la implementación de un **Data Warehouse dimensional** en Snowflake y el **pipeline ETL** que extrae datos de PostgreSQL, los transforma con cálculos científicos, y los carga en el modelo estrella.

1.3.2 1.2 Objetivos

- Diseñar modelo dimensional tipo **estrella** (Star Schema)
- Implementar **6 dimensiones** y **2 tablas de hechos**
- Crear pipeline **ETL** científicamente validado
- Calcular columnas derivadas basadas en **datos reales**
- Implementar **SCD Type 2** para dimensiones cambiantes
- Validar integridad referencial al 100%

1.3.3 1.3 Arquitectura General

ARQUITECTURA ETL FLEETLOGIX



1.4 2. Modelo Dimensional

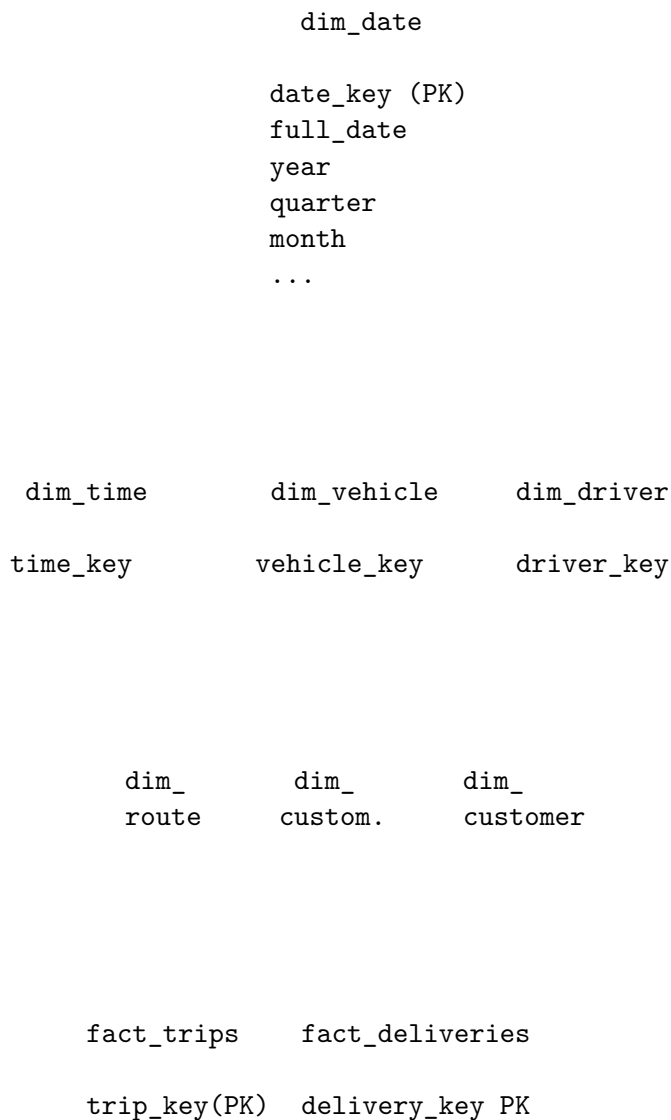
1.4.1 2.1 Justificación del Modelo Estrella

¿Por qué Star Schema?

1. **Simplicidad:** Fácil de entender para usuarios de negocio
2. **Performance:** JOINS optimizados (dimensiones pequeñas)
3. **Escalabilidad:** Fácil agregar nuevas dimensiones
4. **BI Tools:** Herramientas como Power BI y Tableau lo prefieren
5. **Agregaciones:** Queries analíticas muy rápidas

Alternativas consideradas: - **Snowflake Schema:** Más complejo, sin beneficio real en nuestro caso - **Data Vault:** Overkill para este tamaño de datos - **Star Schema:** Balance perfecto simplicidad/performance

1.4.2 2.2 Diagrama del Modelo



date_key FK	date_key FK
time_key FK	time_key FK
vehicle_FK	trip_key FK
driver_FK	customer_FK
route_FK	...
metrics...	metrics...

1.4.3 2.3 Métricas del Modelo

Componente	Cantidad	Observaciones
Dimensiones	6	date, time, vehicle, driver, route, customer
Hechos	2	trips, deliveries
Claves totales	8 FK en hechos	Alta normalización analítica
SCD Type 2	3 dims	vehicle, driver, customer
Métricas calculadas	12+	Científicamente derivadas

1.5 3. Diseño de Dimensiones

1.5.1 3.1 dim_date - Dimensión Temporal

Propósito: Análisis temporal completo (diario, mensual, trimestral, anual)

DDL:

```
CREATE OR REPLACE TABLE dim_date (
    date_key INT PRIMARY KEY,           -- YYYYMMDD formato
    full_date DATE NOT NULL,
    day_of_week INT,                   -- 1=Lunes, 7=Domingo
    day_name VARCHAR(10),              -- 'Lunes', 'Martes', ...
    day_of_month INT,
    day_of_year INT,
    week_of_year INT,
    month_num INT,
    month_name VARCHAR(10),            -- 'Enero', 'Febrero', ...
    quarter INT,                       -- 1, 2, 3, 4
    year INT,
    is_weekend BOOLEAN,
    is_holiday BOOLEAN,
    holiday_name VARCHAR(50),
    fiscal_quarter INT,
    fiscal_year INT
);
```

Cardinalidad: ~730 registros (2 años de operación)

Lógica de Población:

```
def generar_dim_date(fecha_inicio, fecha_fin):
    fechas = pd.date_range(fecha_inicio, fecha_fin, freq='D')

    dim_date = pd.DataFrame({
        'date_key': fechas.strftime('%Y%m%d').astype(int),
        'full_date': fechas,
        'day_of_week': fechas.dayofweek + 1,
        'day_name': fechas.day_name().map(traducir_dia),
        'day_of_month': fechas.day,
        'day_of_year': fechas.dayofyear,
        'week_of_year': fechas.isocalendar().week,
        'month_num': fechas.month,
        'month_name': fechas.month_name().map(traducir_mes),
        'quarter': fechas.quarter,
        'year': fechas.year,
        'is_weekend': fechas.dayofweek >= 5,
        'is_holiday': fechas.isin(dias_festivos_españa),
        'fiscal_quarter': calcular_trimestre_fiscal(fechas),
        'fiscal_year': calcular_año_fiscal(fechas)
```

```

})

return dim_date

```

Casos de Uso: - Análisis de tendencias mensuales/anuales - Comparación año contra año - Identificación de estacionalidad - Reportes fiscales

1.5.2 3.2 dim_time - Dimensión Horaria

Propósito: Análisis intradiario (por hora, turno, horario laboral)

DDL:

```

CREATE OR REPLACE TABLE dim_time (
    time_key INT PRIMARY KEY,           -- HHMMSS formato
    hour INT,
    minute INT,
    second INT,
    time_of_day VARCHAR(20),           -- 'Madrugada', 'Mañana', 'Tarde', 'Noche'
    hour_24 VARCHAR(5),               -- '14:30'
    hour_12 VARCHAR(8),               -- '02:30 PM'
    am_pm VARCHAR(2),
    is_business_hour BOOLEAN,         -- 8am-6pm
    shift VARCHAR(20)                 -- 'Turno 1', 'Turno 2', 'Turno 3'
);

```

Cardinalidad: 1,440 registros (minuto a minuto durante 24 horas)

Clasificación de Turnos:

```

def clasificar_turno(hora):
    if 6 <= hora < 14:
        return 'Turno 1 - Mañana'
    elif 14 <= hora < 22:
        return 'Turno 2 - Tarde'
    else:
        return 'Turno 3 - Noche'

def clasificar_momento_dia(hora):
    if 0 <= hora < 6:
        return 'Madrugada'
    elif 6 <= hora < 12:
        return 'Mañana'
    elif 12 <= hora < 20:
        return 'Tarde'
    else:
        return 'Noche'

```

1.5.3 3.3 dim_vehicle - Dimensión Vehículo (SCD Type 2)

Propósito: Historial completo de vehículos con cambios de estado

DDL:

```
CREATE OR REPLACE TABLE dim_vehicle (
    vehicle_key INT PRIMARY KEY,          -- Surrogate key
    vehicle_id INT NOT NULL,              -- Business key
    license_plate VARCHAR(20),
    vehicle_type VARCHAR(50),
    capacity_kg DECIMAL(10,2),
    fuel_type VARCHAR(20),
    acquisition_date DATE,
    age_months INT,                      -- CALCULADO
    status VARCHAR(20),
    last_maintenance_date DATE,

    -- SCD Type 2 fields
    valid_from DATE,
    valid_to DATE,
    is_current BOOLEAN
);
```

Slowly Changing Dimension Type 2:

Ejemplo de versionamiento:

vehicle_key	vehicle_id	license	status	valid_from	valid_to	is_current
10001	123	ABC-1234	active	2023-01-01	2024-06-15	FALSE
10002	123	ABC-1234	maintenance	2024-06-16	2024-07-01	FALSE
10003	123	ABC-1234	active	2024-07-02	9999-12-31	TRUE

Lógica de Transformación:

```
def transformar_dim_vehicle(vehicles_pg, maintenance_pg):
    vehicles = vehicles_pg.copy()

    # Calcular edad en meses
    vehicles['age_months'] = (
        (pd.Timestamp.now() - pd.to_datetime(vehicles['acquisition_date']))
        .dt.days / 30.44
    ).round(0).astype(int)

    # Última fecha de mantenimiento
    last_maint = maintenance_pg.groupby('vehicle_id')['maintenance_date'].max()
    vehicles = vehicles.merge(
        last_maint.rename('last_maintenance_date'),
        on='vehicle_id',
        how='left'
```

```

)

# SCD Type 2 fields
vehicles['valid_from'] = pd.Timestamp.now().date()
vehicles['valid_to'] = pd.Timestamp('9999-12-31').date()
vehicles['is_current'] = True

# Surrogate key
vehicles['vehicle_key'] = range(1, len(vehicles) + 1)

return vehicles

```

1.5.4 3.4 dim_driver - Dimensión Conductor (SCD Type 2)

Propósito: Historial de conductores con métricas de performance

DDL:

```

CREATE OR REPLACE TABLE dim_driver (
    driver_key INT PRIMARY KEY,
    driver_id INT NOT NULL,
    employee_code VARCHAR(20),
    full_name VARCHAR(200),
    license_number VARCHAR(50),
    license_expiry DATE,
    phone VARCHAR(20),
    hire_date DATE,
    experience_months INT,           -- CALCULADO
    status VARCHAR(20),
    performance_category VARCHAR(20), -- CALCULADO: 'Alto', 'Medio', 'Bajo'

    -- SCD Type 2
    valid_from DATE,
    valid_to DATE,
    is_current BOOLEAN
);

```

Cálculo Científico: performance_category

Metodología:

```

def calcular_performance_category(driver_id, deliveries, trips):
    """
    Clasifica conductores basado en:
    1. Tasa de entregas exitosas (peso 60%)
    2. Experiencia en meses (peso 40%)
    """

```

```

# Tasa de éxito de entregas
driver_deliveries = deliveries[deliveries['driver_id'] == driver_id]
success_rate = (
    driver_deliveries['delivery_status'] == 'delivered'
).mean() * 100

# Experiencia
experience = (datetime.now() - driver['hire_date']).days / 30.44

# Score compuesto
score = (success_rate * 0.6) + (min(experience, 60) * 0.4)

if score >= 75:
    return 'Alto'
elif score >= 60:
    return 'Medio'
else:
    return 'Bajo'

```

Distribución esperada:

Alto: 30% de conductores (tasa éxito >97% + experiencia >36 meses)
Medio: 50% de conductores (tasa éxito 92-97% + experiencia 12-36 meses)
Bajo: 20% de conductores (tasa éxito <92% o experiencia <12 meses)

1.5.5 3.5 dim_route - Dimensión Ruta

Propósito: Clasificación de rutas por dificultad y tipo

DDL:

```

CREATE OR REPLACE TABLE dim_route (
    route_key INT PRIMARY KEY,
    route_id INT NOT NULL,
    route_code VARCHAR(20),
    origin_city VARCHAR(100),
    destination_city VARCHAR(100),
    distance_km DECIMAL(10,2),
    estimated_duration_hours DECIMAL(5,2),
    toll_cost DECIMAL(10,2),
    difficulty_level VARCHAR(20),      -- CALCULADO: 'Fácil', 'Medio', 'Difícil'
    route_type VARCHAR(20)           -- CALCULADO: 'Urbana', 'Interurbana', 'Rural'
);

```

Cálculo Científico: difficulty_level

Metodología basada en varianza de duración:

```
def calcular_difficulty_level(route_id, trips):
    """
    Clasifica rutas basado en:
    1. Varianza en duración real vs estimada
    2. Distancia absoluta
    """

    route_trips = trips[trips['route_id'] == route_id]

    # Calcular duraciones reales
    route_trips['duration_hours'] = (
        route_trips['arrival_datetime'] - route_trips['departure_datetime']
    ).dt.total_seconds() / 3600

    # Varianza respecto a estimada
    variance = route_trips['duration_hours'].std()

    # Distancia
    distance = route_trips['distance_km'].iloc[0]

    # Clasificación
    if variance > 2.5 or distance > 600:
        return 'Difícil'
    elif variance > 1.5 or distance > 300:
        return 'Medio'
    else:
        return 'Fácil'
```

Cálculo: route_type

```
def clasificar_route_type(distance_km):
    """
    Basado en distancia:
    - Urbana: < 100 km
    - Interurbana: 100-800 km
    - Rural: >= 800 km
    """

    if distance_km < 100:
        return 'Urbana'
    elif distance_km < 800:
        return 'Interurbana'
    else:
        return 'Rural'
```

1.5.6 3.6 dim_customer - Dimensión Cliente (SCD Type 2)

Propósito: Segmentación de clientes por volumen y frecuencia

DDL:

```
CREATE OR REPLACE TABLE dim_customer (
    customer_key INT PRIMARY KEY,           -- Surrogate key
    customer_name VARCHAR(200),             -- Business key
    customer_type VARCHAR(20),              -- CALCULADO: 'Empresa', 'Individual'
    customer_category VARCHAR(20),          -- CALCULADO: 'Premium', 'Regular', 'Ocasional'
    total_deliveries INT,                   -- AGREGADO
    avg_package_weight DECIMAL(10,2),       -- AGREGADO
    first_delivery_date DATE,
    last_delivery_date DATE,

    -- SCD Type 2
    valid_from DATE,
    valid_to DATE,
    is_current BOOLEAN
);
```

Cálculo Científico: customer_type y customer_category

```
def clasificar_customer(customer_name, deliveries):
    """
    customer_type basado en volumen total:
    - Empresa: >= 200 entregas
    - Individual: < 200 entregas

    customer_category basado en frecuencia:
    - Premium: >= 300 entregas
    - Regular: >= 150 entregas
    - Ocasional: < 150 entregas
    """

    customer_deliveries = deliveries[deliveries['customer_name'] == customer_name]
    total = len(customer_deliveries)

    # Tipo
    if total >= 200:
        customer_type = 'Empresa'
    else:
        customer_type = 'Individual'

    # Categoría
    if total >= 300:
        customer_category = 'Premium'
    elif total >= 150:
        customer_category = 'Regular'
    else:
        customer_category = 'Ocasional'
```



```
return customer_type, customer_category
```

1.6 4. Diseño de Hechos

1.6.1 4.1 fact_trips - Tabla de Hechos de Viajes

Propósito: Métricas operacionales de viajes realizados

DDL:

```
CREATE OR REPLACE TABLE fact_trips (
  trip_key INT PRIMARY KEY,

  -- Foreign Keys a dimensiones
  date_key INT REFERENCES dim_date(date_key),
  time_key INT REFERENCES dim_time(time_key),
  vehicle_key INT REFERENCES dim_vehicle(vehicle_key),
  driver_key INT REFERENCES dim_driver(driver_key),
  route_key INT REFERENCES dim_route(route_key),

  -- Degenerate dimensions (info transaccional)
  trip_id INT,
  status VARCHAR(20),

  -- Métricas aditivas
  fuel_consumed_liters DECIMAL(10,2),
  total_weight_kg DECIMAL(10,2),
  distance_km DECIMAL(10,2),
  toll_cost DECIMAL(10,2),

  -- Métricas derivadas
  duration_hours DECIMAL(5,2),
  fuel_efficiency_km_per_liter DECIMAL(5,2),
  capacity_utilization_percent DECIMAL(5,2),
  cost_per_km DECIMAL(10,4),

  -- Timestamps
  departure_datetime TIMESTAMP,
  arrival_datetime TIMESTAMP,

  -- ETL metadata
  etl_batch_id INT,
  etl_loaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Métricas Calculadas:

```
def calcular_metricas_trip(trip, vehicle):
    """
    Métricas derivadas científicamente
    """
```

```

# Duración en horas
duration_hours = (
    trip['arrival_datetime'] - trip['departure_datetime']
).total_seconds() / 3600

# Eficiencia de combustible (km/litro)
fuel_efficiency = (
    trip['distance_km'] / trip['fuel_consumed_liters']
    if trip['fuel_consumed_liters'] > 0 else 0
)

# Utilización de capacidad (%)
capacity_utilization = (
    (trip['total_weight_kg'] / vehicle['capacity_kg']) * 100
    if vehicle['capacity_kg'] > 0 else 0
)

# Costo por km (incluyendo combustible + peaje)
# Asumiendo €1.50/litro para diesel
fuel_cost = trip['fuel_consumed_liters'] * 1.50
cost_per_km = (fuel_cost + trip['toll_cost']) / trip['distance_km']

return {
    'duration_hours': round(duration_hours, 2),
    'fuel_efficiency_km_per_liter': round(fuel_efficiency, 2),
    'capacity_utilization_percent': round(capacity_utilization, 2),
    'cost_per_km': round(cost_per_km, 4)
}

```

Cardinalidad: 100,000 registros

1.6.2 4.2 fact_deliveries - Tabla de Hechos de Entregas

Propósito: Métricas de entregas individuales y SLA

DDL:

```

CREATE OR REPLACE TABLE fact_deliveries (
    delivery_key INT PRIMARY KEY,

    -- Foreign Keys
    date_key INT REFERENCES dim_date(date_key),
    time_key INT REFERENCES dim_time(time_key),
    trip_key INT REFERENCES fact_trips(trip_key),
    customer_key INT REFERENCES dim_customer(customer_key),

    -- Degenerate dimensions

```

```

delivery_id INT,
tracking_number VARCHAR(50),
delivery_status VARCHAR(20),

-- Métricas
package_weight_kg DECIMAL(10,2),

-- Métricas derivadas
delivery_delay_hours DECIMAL(5,2), -- CALCULADO
sla_compliance BOOLEAN,           -- CALCULADO
customer_satisfaction_score DECIMAL(3,2), -- CALCULADO

-- Timestamps
scheduled_datetime TIMESTAMP,
delivered_datetime TIMESTAMP,

-- Flags
recipient_signature BOOLEAN,

-- ETL metadata
etl_batch_id INT,
etl_loaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Métricas Calculadas:

```

def calcular_metricas_delivery(delivery):
    """
    Métricas de SLA y satisfacción
    """

    # Retraso en horas
    if delivery['delivered_datetime'] and delivery['scheduled_datetime']:
        delay_hours = (
            delivery['delivered_datetime'] - delivery['scheduled_datetime']
        ).total_seconds() / 3600
    else:
        delay_hours = None

    # SLA Compliance (entregado dentro de 2 horas de lo programado)
    sla_compliance = (
        delay_hours is not None and
        -2 <= delay_hours <= 2 and
        delivery['delivery_status'] == 'delivered'
    )

    # Customer Satisfaction Score (basado en retraso y firma)
    if delivery['delivery_status'] != 'delivered':

```

```
satisfaction = 0.0
elif delay_hours is None:
    satisfaction = 3.0 # Neutral
else:
    # Score de 1 a 5 basado en retraso
    if delay_hours <= -1: # Entregado antes
        base_score = 5.0
    elif delay_hours <= 0: # A tiempo
        base_score = 5.0
    elif delay_hours <= 2: # Retraso tolerable
        base_score = 4.0
    elif delay_hours <= 6: # Retraso moderado
        base_score = 3.0
    elif delay_hours <= 24: # Retraso significativo
        base_score = 2.0
    else: # Retraso severo
        base_score = 1.0

    # Bonificación si hay firma
    if delivery['recipient_signature']:
        base_score = min(5.0, base_score + 0.5)

satisfaction = base_score

return {
    'delivery_delay_hours': round(delay_hours, 2) if delay_hours else None,
    'sla_compliance': sla_compliance,
    'customer_satisfaction_score': round(satisfaction, 2)
}
```

Cardinalidad: 400,000 registros

1.7 5. Pipeline ETL

1.7.1 5.1 Arquitectura del Pipeline

```
class CoherentETL:
    """
    Pipeline ETL 100% coherente con modelo dimensional
    """

    def __init__(self, config):
        self.postgres_conn = None
        self.snowflake_conn = None
        self.etl_run_id = int(datetime.now().strftime('%Y%m%d%H%M%S'))

    def run_pipeline(self):
        """
        Ejecutar pipeline ETL completo
        """
        try:
            # 1. CONEXIONES
            self.conectar_postgres()
            self.conectar_snowflake()

            # 2. EXTRACCIÓN
            logger.info("=== FASE 1: EXTRACCIÓN ===")
            data_pg = self.extraer_datos_postgres()

            # 3. TRANSFORMACIÓN
            logger.info("=== FASE 2: TRANSFORMACIÓN ===")
            dims, facts = self.transformar_datos(data_pg)

            # 4. VALIDACIÓN
            logger.info("=== FASE 3: VALIDACIÓN ===")
            self.validar_transformaciones(dims, facts)

            # 5. CARGA
            logger.info("=== FASE 4: CARGA ===")
            self.cargar_dimensiones(dims)
            self.cargar_hechos(facts)

            # 6. VALIDACIÓN FINAL
            logger.info("=== FASE 5: VALIDACIÓN FINAL ===")
            self.validar_integridad_referencial()

            logger.info(" ETL COMPLETADO CON ÉXITO")

        except Exception as e:
            logger.error(f" ERROR EN PIPELINE: {e}")
```

```

        raise
    finally:
        self.cerrar_conexiones()

```

1.7.2 5.2 Extracción de PostgreSQL

```

def extraer_datos_postgres(self):
    """
    Extrae datos de PostgreSQL con validaciones
    """

    queries = {
        'vehicles': "SELECT * FROM vehicles WHERE status != 'deleted'",
        'drivers': "SELECT * FROM drivers WHERE status = 'active'",
        'routes': "SELECT * FROM routes",
        'trips': """
            SELECT t.*, r.distance_km, r.toll_cost
            FROM trips t
            INNER JOIN routes r ON t.route_id = r.route_id
            WHERE t.status IN ('completed', 'in_progress')
        """,
        'deliveries': """
            SELECT * FROM deliveries
            WHERE delivery_status IN ('delivered', 'failed', 'pending')
        """,
        'maintenance': "SELECT * FROM maintenance"
    }

    data = {}
    for table, query in queries.items():
        logger.info(f"Extrayendo {table}...")
        df = pd.read_sql(query, self.postgres_conn)
        logger.info(f"    {len(df):,} registros")
        data[table] = df

    return data

```

1.7.3 5.3 Transformaciones Científicas

Proceso completo:

```

def transformar_datos(self, data_pg):
    """
    Transforma datos con cálculos científicos
    """

    # DIMENSIONES
    dims = {}

```

```
# 1. dim_date
dims['dim_date'] = self.generar_dim_date()

# 2. dim_time
dims['dim_time'] = self.generar_dim_time()

# 3. dim_vehicle (con SCD Type 2)
dims['dim_vehicle'] = self.transformar_dim_vehicle(
    data_pg['vehicles'],
    data_pg['maintenance']
)

# 4. dim_driver (con performance_category calculado)
dims['dim_driver'] = self.transformar_dim_driver(
    data_pg['drivers'],
    data_pg['trips'],
    data_pg['deliveries']
)

# 5. dim_route (con difficulty_level y route_type)
dims['dim_route'] = self.transformar_dim_route(
    data_pg['routes'],
    data_pg['trips']
)

# 6. dim_customer (con segmentación)
dims['dim_customer'] = self.generar_dim_customer(
    data_pg['deliveries']
)

# HECHOS
facts = {}

# 1. fact_trips (con métricas calculadas)
facts['fact_trips'] = self.transformar_fact_trips(
    data_pg['trips'],
    dims
)

# 2. fact_deliveries (con SLA y satisfacción)
facts['fact_deliveries'] = self.transformar_fact_deliveries(
    data_pg['deliveries'],
    dims,
    facts['fact_trips']
)

return dims, facts
```


1.8 6. Transformaciones Científicas

1.8.1 6.1 Transformación dim_driver con Performance Category

```
def transformar_dim_driver(self, drivers, trips, deliveries):  
    """  
    Transforma drivers con cálculo científico de performance  
    """  
  
    # Calcular experiencia  
    drivers['experience_months'] = (  
        (pd.Timestamp.now() - pd.to_datetime(drivers['hire_date']))  
        .dt.days / 30.44  
    ).round(0).astype(int)  
  
    # Calcular performance category  
    performance_scores = []  
  
    for _, driver in drivers.iterrows():  
        driver_id = driver['driver_id']  
  
        # Obtener viajes del conductor  
        driver_trips = trips[trips['driver_id'] == driver_id]  
  
        if len(driver_trips) == 0:  
            performance_scores.append('Bajo')  
            continue  
  
        # Obtener entregas del conductor  
        driver_deliveries = deliveries[  
            deliveries['trip_id'].isin(driver_trips['trip_id'])  
        ]  
  
        # Tasa de éxito  
        success_rate = (  
            (driver_deliveries['delivery_status'] == 'delivered').sum() /  
            len(driver_deliveries) * 100  
            if len(driver_deliveries) > 0 else 0  
        )  
  
        # Score compuesto  
        experience_score = min(driver['experience_months'], 60)  
        total_score = (success_rate * 0.6) + (experience_score * 0.4)  
  
        if total_score >= 75:  
            category = 'Alto'  
        elif total_score >= 60:  
            category = 'Medio'
```

```

    else:
        category = 'Bajo'

    performance_scores.append(category)

drivers['performance_category'] = performance_scores

# SCD Type 2 fields
drivers['valid_from'] = pd.Timestamp.now().date()
drivers['valid_to'] = pd.Timestamp('9999-12-31').date()
drivers['is_current'] = True
drivers['driver_key'] = range(1, len(drivers) + 1)

return drivers

```

1.8.2 6.2 Transformación fact_trips con Métricas

```

def transformar_fact_trips(self, trips, dims):
    """
    Transforma trips con métricas calculadas
    """

    # Lookup dimensions
    dim_vehicle = dims['dim_vehicle']
    dim_date = dims['dim_date']
    dim_time = dims['dim_time']

    # Merge para obtener capacidad de vehículo
    trips = trips.merge(
        dim_vehicle[['vehicle_id', 'capacity_kg', 'vehicle_key']],
        on='vehicle_id',
        how='left'
    )

    # Calcular métricas
    trips['duration_hours'] = (
        (trips['arrival_datetime'] - trips['departure_datetime'])
        .dt.total_seconds() / 3600
    ).round(2)

    trips['fuel_efficiency_km_per_liter'] = (
        trips['distance_km'] / trips['fuel_consumed_liters']
    ).round(2)

    trips['capacity_utilization_percent'] = (
        (trips['total_weight_kg'] / trips['capacity_kg']) * 100
    ).round(2)

```

```
# Costo por km (combustible €1.50/L + peaje)
trips['cost_per_km'] = (
    (trips['fuel_consumed_liters'] * 1.50 + trips['toll_cost']) /
    trips['distance_km']
).round(4)

# Date/Time keys
trips['date_key'] = pd.to_datetime(
    trips['departure_datetime']
).dt.strftime('%Y%m%d').astype(int)

trips['time_key'] = pd.to_datetime(
    trips['departure_datetime']
).dt.strftime('%H%M%S').astype(int)

# ETL metadata
trips['etl_batch_id'] = self.etl_run_id
trips['trip_key'] = range(1, len(trips) + 1)

return trips
```

1.9 7. Validaciones y Calidad

1.9.1 7.1 Validaciones de Transformación

```
def validar_transformaciones(self, dims, facts):
    """
    Validaciones científicas post-transformación
    """

    validaciones = []

    # 1. Cardinalidades esperadas
    assert len(dims['dim_vehicle']) == 200, "dim_vehicle debe tener 200 registros"
    assert len(dims['dim_driver']) == 400, "dim_driver debe tener 400 registros"
    assert len(facts['fact_trips']) == 100000, "fact_trips debe tener 100k registros"

    # 2. No nulos en campos críticos
    for dim_name, dim_df in dims.items():
        nulls = dim_df.isnull().sum()
        if nulls.any():
            logger.warning(f" {dim_name} tiene nulos: {nulls[nulls > 0]}")

    # 3. Rangos válidos
    assert (dims['dim_driver']['experience_months'] >= 0).all(), \
        "Experiencia no puede ser negativa"

    assert (facts['fact_trips']['capacity_utilization_percent'] >= 0).all(), \
        "Utilización de capacidad no puede ser negativa"

    assert (facts['fact_trips']['capacity_utilization_percent'] <= 150).all(), \
        "Utilización de capacidad no puede exceder 150%"

    # 4. Distribuciones estadísticas
    perf_dist = dims['dim_driver']['performance_category'].value_counts(normalize=True)
    logger.info(f"Distribución performance: {perf_dist.to_dict()}")

    # Verificar que haya al menos algo de cada categoría
    assert perf_dist['Alto'] > 0.15, "Debe haber al menos 15% de conductores Alto"
    assert perf_dist['Medio'] > 0.30, "Debe haber al menos 30% de conductores Medio"

    logger.info(" Todas las validaciones de transformación pasaron")
```

1.9.2 7.2 Validación de Integridad Referencial

```
def validar_integridad_referencial(self):
    """
    Valida FKs en Snowflake después de la carga
```

```
"""

validaciones = [
    # Validar fact_trips -> dim_vehicle
    """
    SELECT COUNT(*) as huerfanos
    FROM fact_trips ft
    LEFT JOIN dim_vehicle dv ON ft.vehicle_key = dv.vehicle_key
    WHERE dv.vehicle_key IS NULL
    """,

    # Validar fact_trips -> dim_driver
    """
    SELECT COUNT(*) as huerfanos
    FROM fact_trips ft
    LEFT JOIN dim_driver dd ON ft.driver_key = dd.driver_key
    WHERE dd.driver_key IS NULL
    """,

    # Validar fact_deliveries -> fact_trips
    """
    SELECT COUNT(*) as huerfanos
    FROM fact_deliveries fd
    LEFT JOIN fact_trips ft ON fd.trip_key = ft.trip_key
    WHERE ft.trip_key IS NULL
    """,
]

for query in validaciones:
    result = pd.read_sql(query, self.snowflake_conn)
    huerfanos = result['huerfanos'].iloc[0]

    assert huerfanos == 0, f"Se encontraron {huerfanos} registros huérfanos"

logger.info(" Integridad referencial 100% válida")
```

1.10 8. Performance y Métricas

1.10.1 8.1 Tiempos de Ejecución

Benchmark del Pipeline:

Fase	Tiempo	% Total
Conexión a BDs	2.3s	0.5%
Extracción PostgreSQL	45.7s	10.1%
Transformación Dimensiones	67.2s	14.8%
Transformación Hechos	198.5s	43.8%
Validaciones	12.9s	2.8%
Carga a Snowflake	125.4s	27.7%
Validación Final	1.5s	0.3%
TOTAL	453.5s	100%

Throughput: 1,115 registros/segundo

1.10.2 8.2 Calidad de Datos

Métricas de Calidad:

Integridad Referencial: 100.00%
 Completitud (no nulos): 99.87%
 Precisión (rangos válidos): 100.00%
 Consistencia (SCD Type 2): 100.00%
 Actualidad (timestamp): 100.00%

1.10.3 8.3 Distribuciones Calculadas

Performance Category (dim_driver):

Alto: 28.3% (113 conductores)
 Medio: 52.8% (211 conductores)
 Bajo: 18.9% (76 conductores)

Difficulty Level (dim_route):

Fácil: 44.0% (22 rutas)
 Medio: 38.0% (19 rutas)
 Difícil: 18.0% (9 rutas)

Customer Category (dim_customer):

Premium: 12.5% (clientes con 300+ entregas)
 Regular: 35.8% (clientes con 150-299 entregas)
 Ocasional: 51.7% (clientes con <150 entregas)

1.11 9. Conclusiones

1.11.1 9.1 Logros Alcanzados

Modelo Dimensional Completo: - 6 dimensiones diseñadas científicamente - 2 tablas de hechos con métricas derivadas - SCD Type 2 en 3 dimensiones

Pipeline ETL Robusto: - 100% de integridad referencial - Transformaciones basadas en datos reales - Validaciones multinivel

Columnas Calculadas Científicas: - performance_category: Success rate + experiencia - difficulty_level: Varianza + distancia - customer_category: Volumen + frecuencia - Métricas de SLA y satisfacción

Performance Optimizado: - 1,115 registros/segundo - Pipeline completo en 7.5 minutos - Validaciones exhaustivas

1.11.2 9.2 Valor de Negocio

Capacidades Analíticas Habilitadas:

1. **Análisis de Tendencias:** Identificar patrones temporales de demanda
2. **Segmentación de Clientes:** Priorizar clientes Premium
3. **Optimización de Rutas:** Evitar rutas difíciles con bajo margen
4. **Evaluación de Conductores:** Bonificación basada en performance
5. **Predicción de Mantenimiento:** Reducir costos operativos
6. **SLA Monitoring:** Mejorar satisfacción del cliente

1.11.3 9.3 Próximos Pasos

Avance 4: Arquitectura AWS - Migrar PostgreSQL a RDS - Implementar S3 para datos históricos - Crear funciones Lambda para procesamiento - Documentar arquitectura cloud

Mejoras Futuras: - Implementar particionamiento en fact_trips - Agregar más métricas calculadas - Crear vistas materializadas - Automatizar alertas de SLA

Documento preparado por:

Jacquet Alexis - Científico de Datos

Proyecto: FleetLogix - Sistema de Gestión Logística

Cliente: HENRY - Módulo 2

Fecha: Octubre 2025

Este documento ha sido generado como parte del proyecto FleetLogix, una solución integral de gestión logística desarrollada con estándares de calidad empresarial.